

Génie logiciel

Livrable 1

CESI 
ÉCOLE D'INGÉNIEURS



ProSoft

Génie Logiciel

Réalisé par

Maéva CHALLIES

Allan BOUHAMED

Enzo CADIÈRE

STOFFEL Maxime

Mathéo CARVAL

Philippe LUU

Projet encadré par

Ali SKAF

Année universitaire 2025-2026

Livrable 1 - 06/02/2026

Informations générales

Versions du document :

Ce tableau retrace toutes les versions du document :

Version	Date	Auteur(s)	Objet
1.0	01/2026	LUU Philippe, CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo, STOFFEL Maxime, BOUHAMED Allan	Création du document
1.1	02/2026	CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo	Revue du document et des parties : 3.1, 3.4, 5.3

Liste de diffusion :

Ce tableau répertorie tous les destinataires ayant accès au document :

Entreprise / Organisme	Destinataire(s)	Contacts	Rôle
ProSoft	LUU Philippe	philippe.luu@prosoft.fr	Ingénieur Chef de projet & Scrum Master
ProSoft	CARVAL Matheo	matheo.carval@prosoft.fr	Ingénieur Architecte développement (Lead Dev)
ProSoft	CADIÈRE Enzo	enzo.cadiere@prosoft.fr	Ingénieur DevOps & CI
ProSoft	CHALLIES Maéva	maeva.challies@prosoft.fr	Ingénieure Qualité & Product Owner (Co-Lead Dev)
ProSoft	BOUHAMED Allan	allan.bouhamed@prosoft.fr	Ingénieur d'Études et Développement C#
ProSoft	STOFFEL Maxime	maxime.stoffel@prosoft.fr	Ingénieur d'Études et Développement C#
CESI	AOUAM Djamel	daouam@cesi.fr	Directeur informatique
CESI	SKAF Ali	askaf@cesi.fr	Responsable informatique

Synthèse Managériale

Ce document présente le bilan technique et fonctionnel de la version 1.0 du logiciel de sauvegarde EasySave, développé par l'équipe d'ingénierie logicielle pour le compte de ProSoft.

Double objectif pour ce premier livrable : fournir un moteur de sauvegarde robuste et performant (Console / .NET 8) tout en établissant une architecture logicielle évolutive capable de supporter les futures interfaces graphiques et le parallélisme.

Les points clés de ce livrable sont la conformité technique. Le logiciel respecte l'intégralité du cahier des charges (sauvegardes séquentielles, logs JSON, multilinguisme).

L'architecture pérenne, l'adoption précoce d'une structure modulaire (séparation Core/Console) et de patrons de conception standards (Singleton, Strategy) garantit une transition fluide vers la version 1.1 puis 2.0 (.NET MAUI) sans refonte du code métier.

La maîtrise des coûts, le projet a été livré dans les délais impartis. L'analyse financière estime le coût de R&D à environ 39 000 €, avec un retour sur investissement (ROI) atteignable dès la 197ème licence vendue, confirmant la viabilité économique du produit.

Également, la qualité & maintenance, le respect strict des normes de codage (DRY, Anglais) et la documentation technique fournie assurent une maintenabilité optimale et une reprise aisée par n'importe quelle équipe de ProSoft.

En conclusion, EasySave version 1.0 constitue un socle technologique stable et validé, prêt pour le déploiement des fonctionnalités avancées des prochains jalons.

Sommaire

Informations générales.....	3
Versions du document :	3
Liste de diffusion :	3
Synthèse Managériale.....	4
Sommaire.....	5
1. Contexte et présentation.....	6
1.1 Présentation de ProSoft et du projet EasySave.....	6
1.2 Objectifs & contraintes (cahier des charges).....	6
1.3 Politique tarifaire et maintenance.....	8
2. Gestion de projet et environnement.....	9
2.1 Environnement de développement (Visual Studio 2022, .NET 8.0).....	9
2.2 Stratégie de versioning.....	9
2.3 Workflow de collaboration (GitFlow & Traçabilité des versions.....	10
2.4 Conventions de codage et qualité logicielle (DRY & Anglais).....	11
2.5 Planification temporelle et chiffrage du projet.....	12
2.6 Stratégie de tests unitaires.....	14
2.7 Chaîne CI/CD.....	15
3. Architecture logistique & choix techniques.....	16
3.1 Architecture globale (passage en V2).....	16
3.2 Implémentation des Design Patterns (Singleton & MVVM).....	17
3.2.1 Singleton Pattern (ConfigurationManager).....	17
3.2.2 Strategy Pattern (ILogFormatter).....	17
3.2.3 Dependency Injection Pattern.....	17
3.2.4 Repository Pattern (BackupManager).....	18
3.2.5 Observer Pattern (StateWriter).....	18
3.2.6 Factory Method Pattern (GetFormatter dans LoggerBase).....	18
3.3 Bibliothèque EasyLog.dll (documentation et rétrocompatibilité).....	19
3.4 Gestion des données (System.Text.Json et structuration des fichiers dans %AppData%).....	20
3.5 Internationalisation (gestion du bilinguisme FR/EN dans la console).....	21
4. Spécifications fonctionnelles (V1.0).....	22
4.1 Gestion des travaux de sauvegarde (Limitation à 5, paramètres Source/Cible/Type)...	22
4.2 Exécution et lignes de commande (arguments 1-3, 1;3).....	23
4.3 Journalisation et état en temps réel (Format des fichiers JSON, lisibilité Notepad)...	24
5. Conception (Diagrammes UML).....	26
5.1 Diagramme de cas d'utilisation (Vue macro des fonctionnalités).....	26
5.2 Diagramme de classes (Structure de l'application et de la DLL).....	27
5.3 Diagramme de séquence (Focus sur le processus de sauvegarde et logs en temps réel).....	29

5.4 Diagramme d'activité (Logique de l'exécution séquentielle).....	30
6. Table des figures.....	32
7. Glossaire.....	33
8. Annexes.....	35
8.1 Guide d'installation et Support Technique (Détails ProSoft).....	35
8.2 Lien vers le dépôt GitHub.....	35

1. Contexte et présentation

1.1 Présentation de ProSoft et du projet EasySave

ProSoft est un éditeur de logiciels reconnu, spécialisé dans le développement de solutions informatiques professionnelles. Dans le cadre de l'élargissement de son portefeuille de produits, l'entreprise lance le projet "EasySave", une solution logicielle de sauvegarde destinée à une clientèle B2B (Business to Business).

ProSoft a pour ambition de fournir un outil fiable, performant et simple d'utilisation pour sécuriser les données critiques des entreprises. Le logiciel EasySave doit permettre la gestion de sauvegardes complètes et différentielles, tout en assurant une traçabilité rigoureuse des opérations via des journaux d'événements. Ce projet s'inscrit dans une démarche évolutive, commençant par une version console (V1.0) pour valider le moteur de sauvegarde, avant d'évoluer vers une interface graphique moderne (V2.0) et des fonctionnalités avancées de parallélisme (V3.0).

1.2 Objectifs & contraintes (cahier des charges)

Le mandat confié à l'équipe de développement porte sur la réalisation de la version 1.0 d'EasySave, une application Console opérant sous l'environnement .NET. Le cœur fonctionnel du logiciel doit permettre la configuration et l'orchestration de cinq travaux de sauvegarde distincts.

Chaque travail est défini par un nom unique, un répertoire source et un répertoire cible, pouvant être situés indifféremment sur des disques locaux, des périphériques externes ou des lecteurs réseaux. Le système doit impérativement gérer deux stratégies de sauvegarde, à savoir la sauvegarde complète et la sauvegarde différentielle, tout en garantissant que

l'intégralité des éléments du répertoire source, incluant fichiers et sous-répertoires, soit traitée de manière récursive.

En termes d'utilisabilité, l'application doit être accessible nativement aux utilisateurs anglophones et francophones. Outre l'interaction via le menu, le logiciel doit offrir une interface en ligne de commande permettant l'automatisation des tâches. L'interpréteur de commandes doit prendre en charge des syntaxes spécifiques telles que l'exécution d'une plage de travaux, par exemple " EasySave.exe 1-3 ", ou l'exécution d'une sélection discontinue, par exemple " EasySave.exe 1;3 ".

Une exigence impose le développement de la fonctionnalité de journalisation au sein d'une bibliothèque externe nommée " EasyLog.dll ", conçue pour être réutilisable dans d'autres projets et compatible avec les évolutions futures. Cette bibliothèque doit générer en temps réel un fichier de log journalier au format JSON. Chaque entrée de ce journal doit contenir un horodatage précis, le nom de la sauvegarde, les adresses complètes des fichiers source et destination au format UNC, la taille du fichier ainsi que le temps de transfert exprimé en millisecondes, ce dernier devant apparaître comme une valeur négative en cas d'erreur.

Parallèlement au journal historique, le logiciel doit maintenir un fichier d'état unique, également au format JSON, reflétant la progression en temps réel des travaux. Ce fichier doit recenser l'appellation du travail, l'horodatage de la dernière action et l'état du job, tel que Actif ou Non Actif. Lorsqu'un travail est en cours d'exécution, le fichier d'état doit détailler le nombre total de fichiers éligibles, la taille totale à transférer, le pourcentage de progression, ainsi que le nombre et la taille des fichiers restants, tout en précisant l'adresse complète du fichier source et du fichier de destination actuellement en cours de traitement.

Enfin, des contraintes strictes de déploiement et de formatage s'appliquent. L'utilisation de répertoires temporaires volatiles de type " c:\temp\ " est formellement proscrite au profit d'emplacements persistants compatibles avec des environnements serveurs. Pour assurer la maintenabilité et le débogage, les fichiers JSON générés, tant pour les logs que pour l'état, doivent inclure des retours à la ligne et une indentation permettant une lecture aisée via le Bloc-notes. L'ensemble de l'architecture doit être conçu dès cette version initiale pour faciliter la transition future vers une interface graphique basée sur le modèle MVVM, anticipant ainsi les besoins de la version 2.0.

1.3 Politique tarifaire et maintenance

EasySave n'est pas seulement un défi technique, c'est un produit commercial soumis à une logique de rentabilité précise. La stratégie de tarification fixée par ProSoft établit un prix unitaire de 200 € HT par licence. À cela s'ajoute un contrat de maintenance facturé annuellement à hauteur de 12 % du prix d'achat, incluant les mises à jour mineures et majeures. Le support technique étant assuré sur une plage horaire standard de 8h à 17h, cinq jours sur sept, il est essentiel de fournir une documentation technique et utilisateur irréprochable, ainsi que des fichiers de logs précis. Cette rigueur a pour objectif de réduire le temps de traitement des tickets support et d'augmenter la marge sur le contrat de support.

2. Gestion de projet et environnement

2.1 Environnement de développement (Visual Studio 2022, .NET 8.0)

L'intégralité de l'équipe a orchestré son cadre de travail autour de l'IDE Visual Studio 2022, garantissant une synchronisation parfaite des fichiers de solution (.sln) et de projets (.csproj). Ce choix permet d'exploiter les outils d'analyse statique de code intégrés, essentiels pour respecter la contrainte de non-redondance (DRY) imposée par la direction technique. La solution est structurée de manière modulaire, séparant l'application console du moteur de journalisation EasyLog.dll, développé en tant que bibliothèque de classes distincte pour en assurer la réutilisabilité.

Le projet s'appuie sur le framework .NET 8.0, choisi pour son statut de version LTS (Long Term Support). Cette version garantit à ProSoft une stabilité et des mises à jour de sécurité sur le long terme, tout en offrant des performances optimales pour les opérations d'entrée/sortie (I/O) lors des sauvegardes. L'utilisation de .NET 8 facilite également la transition future vers une interface graphique (MAUI) grâce à une séparation nette entre la logique métier et les interfaces de présentation.

Pour la gestion des dépendances, nous utilisons le gestionnaire de paquets NuGet, notamment pour l'intégration de System.Text.Json, privilégié pour sa légèreté et sa rapidité de traitement des fichiers de configuration et de logs. Enfin, la conception technique a été documentée via PlantUML (UML-as-Code), permettant d'intégrer les schémas directement dans le flux de travail Git. L'intégration native de GitHub dans Visual Studio nous assure un versioning en temps réel et une collaboration fluide, tandis que la compatibilité avec le CLI .NET garantit que le projet reste compilable et testable sur n'importe quel environnement de serveur client sans dépendre exclusivement d'une interface graphique de développement.

2.2 Stratégie de versioning

L'évolution du logiciel EasySave est encadrée par une politique rigoureuse de Semantic Versioning (MAJOR.MINOR.PATCH), garantissant une traçabilité totale et une lecture immédiate de la nature des changements. Pour ce premier livrable, le projet est

officiellement marqué comme la version 1.0.0. Cette structure permet d'indiquer clairement si une modification introduit une rupture de compatibilité (Major), une nouvelle fonctionnalité (Minor), ou une simple correction de bug (Patch), ce qui est essentiel pour la maintenance à long terme demandée par ProSoft.

Le suivi des évolutions repose sur un fichier CHANGELOG.md maintenu à la racine du dépôt GitHub. Ce document constitue la référence officielle de l'historique du projet, décrivant de manière compréhensible les changements apportés entre chaque version. En complément, nous appliquons une convention de commits standardisée (types feat, fix, chore, doc) qui rend l'historique de développement parfaitement lisible. Cette nomenclature n'est pas qu'esthétique, elle permet de lier chaque modification de code à un besoin spécifique du cahier des charges.

Enfin, la gestion technique des versions s'appuie sur des tags Git annotés, créant des points de référence immuables pour chaque release. Cette cohérence entre le code source, les tags et la documentation assure que n'importe quelle équipe de ProSoft pourra reprendre le projet avec une vision claire de son état d'avancement.

2.3 Workflow de collaboration (GitFlow & Traçabilité des versions

La gestion des développements repose sur le modèle GitFlow, structuré autour de deux branches permanentes : la branche main, garante de la stabilité du code livré, et la branche develop, dédiée à l'intégration des fonctionnalités en cours de validation. Pour garantir une isolation totale des tâches et éviter les conflits de fusion, chaque membre de l'équipe travaille sur des branches de fonctionnalités spécifiques (feature branches). Ce cloisonnement permet de ne jamais compromettre la branche principale et assure que chaque ajout est testé avant d'être fusionné.

L'usage du dépôt est caractérisé par une fréquence de "push" régulière, intervenant dès la finalisation de chaque sous-tâche validée par l'équipe. Ce rythme de travail permet au tuteur de suivre l'évolution réelle du projet jour après jour. Avant chaque fusion vers la branche develop, une revue interne est effectuée afin de s'assurer que le code respecte les standards de qualité, notamment l'utilisation de l'anglais et l'absence de redondance.

La traçabilité, importante pour la direction du projet, est assurée par l'application systématique de tags Git annotés lors de chaque jalon important (ex: v1.0-Livable1). Contrairement à une simple branche de version, l'usage des tags permet de figer l'état du code à un instant T, rendant les versions livrées immuables et facilement trouvables par le jury. Cette organisation, couplée à des messages de commits explicites, permet un historique de développement clair où chaque modification est rattachée à une fonctionnalité précise du cahier des charges.

2.4 Conventions de codage et qualité logicielle (DRY & Anglais)

Afin de garantir une exploitation fluide par les filiales internationales de ProSoft, l'intégralité du code source incluant les noms de classes, de méthodes, de variables ainsi que les commentaires et la documentation technique est rédigée exclusivement en anglais. Pour concilier cette exigence technique avec le besoin de bilinguisme de l'interface utilisateur (Français/Anglais), nous avons implémenté un système de gestion de ressources indexé. L'application récupère dynamiquement les chaînes de caractères selon la langue choisie par l'utilisateur, séparant ainsi strictement la logique métier anglophone de l'affichage multilingue.

Afin d'éviter la redondance, le projet est assuré par une application stricte du principe DRY (Don't Repeat Yourself). Pour ce faire, nous avons centralisé les traitements récurrents au sein de méthodes génériques et de classes utilitaires, notamment via la bibliothèque EasyLog.dll qui mutualise la logique de journalisation pour l'ensemble du projet. Nous suivons rigoureusement les standards de nommage Microsoft C#, utilisant le PascalCase pour les types et le camelCase pour les variables locales, tout en privilégiant des identifiants explicites pour faciliter la reprise du code par d'autres équipes.

Enfin, bien qu'aucune limite de lignes arbitraires n'ait été imposée par fonction, nous veillons à ce que chaque unité de code conserve une taille raisonnable et une responsabilité unique pour en optimiser la lisibilité. La qualité globale de la solution est maintenue grâce à l'usage systématique d'un linter pour le respect des normes syntaxiques. Ces outils nous permettent de détecter précocement d'éventuels dysfonctionnements et de garantir une architecture robuste, capable de réagir rapidement en cas d'évolution ou de maintenance corrective.

2.5 Planification temporelle et chiffrage du projet

La maîtrise des délais et des coûts étant un impératif pour la rentabilité du projet EasySave, le pilotage stratégique a été matérialisé par l'élaboration et le suivi rigoureux d'un diagramme de Gantt. Cet outil nous a permis de visualiser l'enchaînement logique des tâches, d'identifier le chemin critique et d'orchestrer la parallélisation des développements entre les différents binômes techniques, garantissant ainsi le respect strict du jalon de livraison de la version 1.0 imposé par ProSoft.

En complément, la gestion opérationnelle quotidienne s'est appuyée sur l'application de la méthode Agile Scrum. L'équipe a mis en place un tableau Kanban numérique via l'outil Trello afin de suivre l'état d'avancement des tâches en temps réel (À faire, En cours, Terminé). Ce suivi a été rythmé par des points de synchronisation journaliers ("Daily Stand-ups"), permettant d'identifier immédiatement les points de blocage et de réajuster la charge de travail de manière dynamique.

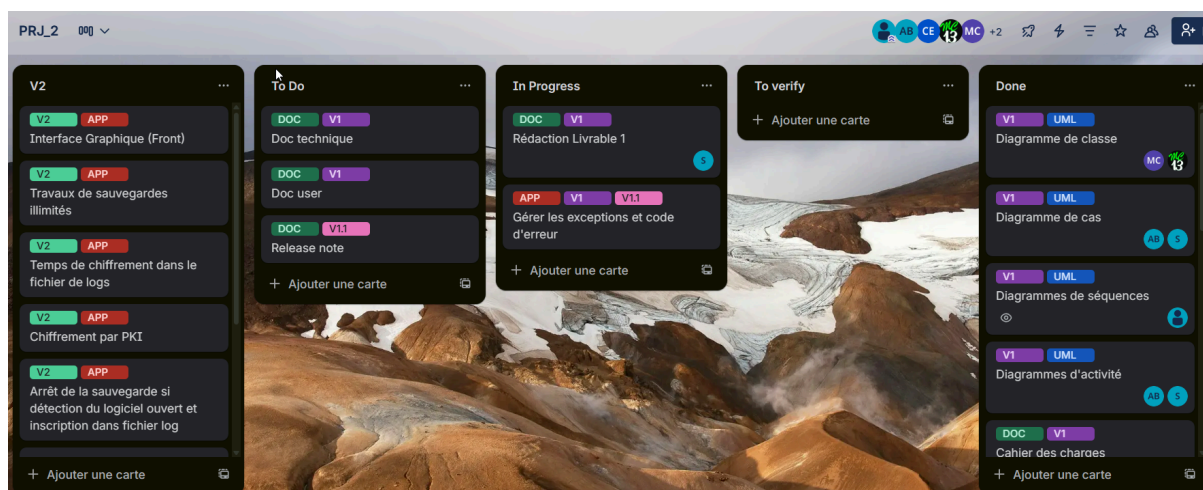


Figure 1 : Trello du projet, avec les cartes usuelles (To Do, In Progress, To Verify et Done) mais dans le To Do, ne figurent que les livrables de la phase 1 (V1 et V1.1), des étiquettes "APP" pour la tâche de développement et "DOC" pour la rédaction des documents (utilisateurs, commentaires) et "UML" pour les diagrammes. Les tâches de la V2 ont été ajoutées en anticipation de la suite.

Une attention particulière a été portée à la gestion des aléas en cours de projet. Suite à une corruption critique de la structure du dépôt GitHub menaçant l'intégrité du versioning, nous

avons opéré un redéploiement immédiat des ressources humaines. Une réunion de crise a acté la réassignation d'Enzo, initialement prévu sur les Managers, à la reconstruction de l'infrastructure Git afin d'assurer la continuité de la collaboration pour les livrables futurs. Pour compenser ce changement, Maeva, dont l'avancement était supérieur aux prévisions, a repris le développement des managers (Configuration et Backup). Cette agilité dans la gestion des ressources a permis de neutraliser l'impact de l'incident sans altérer la date de livraison finale.

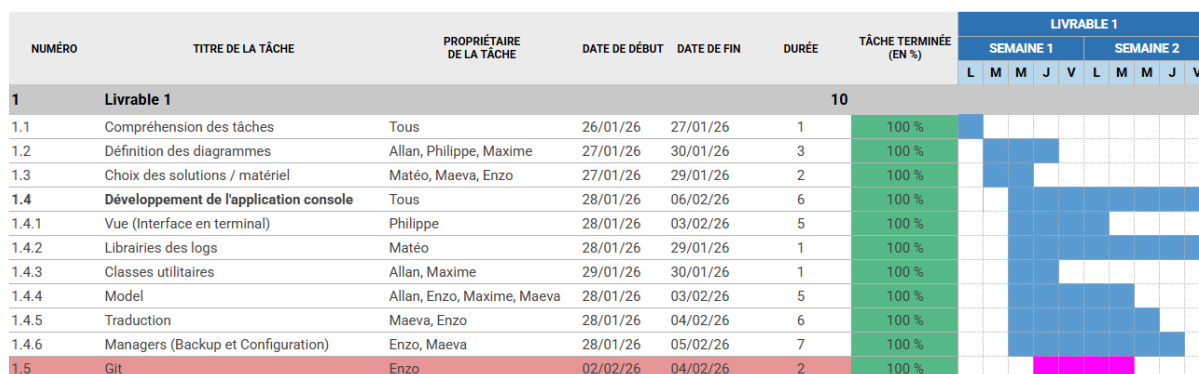


Figure 2 : Diagramme de Gantt du Livrable 1, en bleu l'avancement du projet conforme à nos attentes, en violet un événement imprévu nous ayant amené revoir et paralléliser une tâche.

Au-delà de la gestion temporelle, une dimension économique a été intégrée au suivi de projet via le chiffrage du coût de développement (R&D). Cette valorisation financière repose sur le calcul précis du volume horaire consommé par l'ensemble de l'équipe, pondéré par un coût horaire chargé (incluant salaire et charges patronales) représentatif d'ingénieurs débutants. Cette démarche permet à la direction de ProSoft d'évaluer le coût de revient exact de la version 1.0 et de calculer le retour sur investissement (ROI) nécessaire pour justifier la politique tarifaire.

Entité	Coût entreprise	Volume	Coût total
Ingénieur développement C# (x6)	18.68€/heure	350h	6538 x 6 (Ingénieurs) = 39 228€
Licences EasySave	200€/licence	x	200€

Figure 3 : Tableau de chiffrage des coûts pour l'entreprise cliente

Sur la base de ce chiffrage de développement estimé à 39 228 €, et considérant un prix de vente unitaire de 200 € HT fixé par la politique tarifaire, le seuil de rentabilité du projet (Point Mort) sera atteint dès la vente de la 197ème licence. Ce volume de vente, réaliste au regard du marché visé par ProSoft, confirme la viabilité économique du projet EasySave et la pertinence du modèle économique basé sur les contrats de maintenance.

2.6 Stratégie de tests unitaires

La qualité et la maintenabilité du moteur de sauvegarde constituent un point central du projet EasySave, puisque l'objectif de la version 1.0 est de livrer un socle fonctionnel robuste destiné à évoluer vers des versions plus complexes. Dans cette optique, une stratégie de tests unitaires a été mise en place afin de valider systématiquement le comportement du backend, indépendamment de l'interface console.

Les tests unitaires couvrent l'ensemble des fonctionnalités principales attendues du moteur de sauvegarde. Ils permettent de vérifier la cohérence des règles de gestion des travaux (création, modification, suppression, limite de tâches), ainsi que les scénarios d'exécution, incluant la distinction entre sauvegarde complète et différentielle. La logique de parsing des arguments de ligne de commande fait également partie du périmètre testé, de manière à garantir la conformité aux syntaxes demandées et la robustesse face aux entrées invalides. Enfin, des tests sont dédiés à la production des informations techniques générées en temps réel, notamment la journalisation et l'état d'avancement, afin de sécuriser la traçabilité exigée par le cahier des charges.

Cette couverture est particulièrement importante dans une démarche d'évolution continue, elle permet d'anticiper les régressions lors des versions ultérieures, en garantissant que les

modifications apportées au code métier n'altèrent pas les comportements validés en version 1.0. La conception orientée injection de dépendances facilite cette stratégie, car elle rend possible l'utilisation de doubles de tests et l'isolation des composants critiques (accès fichiers, persistance, formatage). Ainsi, le moteur de sauvegarde peut être validé de manière fiable, sans dépendre des interactions utilisateur.

2.7 Chaîne CI/CD

Afin de garantir la stabilité du code tout au long du projet et d'industrialiser la production des livrables, l'équipe a mis en place une chaîne d'intégration et de déploiement continu (CI/CD) via GitHub Actions. Cette automatisation répond à une exigence implicite de professionnalisation : chaque modification du dépôt doit être contrôlée, reproductible et traçable, afin de réduire les risques et les coûts de maintenance.

La partie CI est exécutée à chaque push et lors des pull requests. Elle applique d'abord un contrôle strict de la qualité de code, notamment via la vérification du formatage et des règles de style. Le projet est ensuite compilé en configuration Release, ce qui permet de détecter précocement des erreurs de build ou de configuration. Les tests unitaires sont alors exécutés automatiquement, et leurs résultats sont publiés afin de fournir une visibilité immédiate sur l'état du backend après chaque évolution. Enfin, un contrôle des dépendances NuGet est appliqué afin de détecter d'éventuelles vulnérabilités connues, y compris dans les dépendances transitives, ce qui renforce la fiabilité globale du produit livré.

La partie CD est conçue pour produire un livrable exploitable et versionné. Elle est déclenchée sur la branche principale, ou manuellement lorsque cela est nécessaire. Avant publication, la pipeline rejoue les tests unitaires, garantissant qu'aucune release ne puisse être générée sur une base instable. Les binaires sont ensuite construits sous forme de publications autoportées et en fichier unique, ce qui facilite la distribution du logiciel dans des environnements clients variés. Le processus automatise également le versioning via la génération de tags et de notes de version à partir de l'historique Git, assurant une traçabilité complète entre le code source, la release publiée et les changements fonctionnels associés.

Cette organisation CI/CD constitue un levier concret de réduction des risques, elle formalise un contrôle qualité continu, assure la reproductibilité des livraisons, et garantit que les prochaines évolutions (1.1, 2.0, 3.0) pourront s'appuyer sur une base de développement stable et industrialisée.

3. Architecture logistique & choix techniques

3.1 Architecture globale (passage en V2)

Bien que la version 1.0 soit opérée via une interface en ligne de commande, nous avons anticipé les exigences techniques de la version 2.0 en adoptant dès le départ une architecture modulaire stricte. Notre structure de code isole la logique métier de la couche de présentation, garantissant que le moteur de sauvegarde n'a aucune dépendance envers la console système. Cette séparation des responsabilités est assurée par l'utilisation de namespaces distincts et une organisation rigoureuse des classes, empêchant tout impact du code métier par des instructions d'affichage directes.

Le cœur du système repose sur la classe BackupManager, qui agit en tant qu'orchestrateur central. Cette classe encapsule l'intelligence de l'application, elle gère le cycle de vie des travaux de sauvegarde, manipule les listes de fichiers et met à jour les états, sans jamais interagir directement avec l'utilisateur. La couche "Vue Console" se contente d'instancier ce gestionnaire et d'afficher les données qu'il retourne. Cette conception garantit une transition fluide et à moindre coût vers le pattern MVVM (Model-View-ViewModel) imposé pour la version 2.0.

Concrètement, lors de l'évolution vers l'interface graphique MAUI, nous pourrions conserver l'intégralité de notre logique métier actuelle. La couche de présentation console sera remplacée par des vues XAML, et notre classe BackupManager sera injectée dans les futurs ViewModels. Ces derniers agiront comme des adaptateurs, transformant les données brutes du gestionnaire en propriétés observables pour l'interface graphique, sans qu'il soit nécessaire de réécrire les algorithmes de sauvegarde ou de journalisation déjà validés en version 1.0.

3.2 Implémentation des Design Patterns (Singleton & MVVM)

3.2.1 Singleton Pattern (ConfigurationManager)

Pour la gestion de la configuration de l'application, nous avons implémenté le pattern Singleton dans la classe ConfigurationManager afin de garantir un point d'accès unique et cohérent aux paramètres globaux de l'application (langue, format de log, chemins de fichiers). Cette implémentation assure qu'une seule instance de configuration existe durant toute la durée de vie de l'application, évitant ainsi les incohérences qui surviendraient si différentes parties du programme utilisaient des configurations contradictoires. L'unicité de l'instance garantit également que les modifications de paramètres (changement de langue via l'interface) se propagent instantanément à tous les composants qui en dépendent, sans nécessiter de mécanisme complexe de synchronisation.

3.2.2 Strategy Pattern (ILogFormatter)

Pour la sérialisation des données de journalisation, nous avons implémenté le pattern Strategy à travers l'interface ILogFormatter<T> et son implémentation concrète JsonFormatter<T> dans la bibliothèque EasyLog.dll. Cette architecture permet de modifier dynamiquement le format de sortie des logs sans altérer la logique du LoggerBase, répondant ainsi à l'exigence du cahier des charges de supporter plusieurs formats d'exportation. Le choix du formatter s'effectue à l'exécution via la méthode GetFormatter<T>() qui instancie automatiquement le formateur approprié selon l'énumération LogFormat configurée, permettant une extension future vers d'autres formats (XML, CSV, binaire) sans modification du code client.

3.2.3 Dependency Injection Pattern

Le projet utilise extensivement le pattern Dependency Injection dans toutes les classes principales pour découpler les dépendances et faciliter la testabilité. Le BackupManager reçoit via son constructeur un FileTransferService et un StateWriter, le FileTransferService reçoit un ILogger et un StateWriter, la ConsoleUI reçoit un LocalizationService et un BackupManager. Cette approche respecte le principe d'inversion de dépendances (D du SOLID) en permettant l'injection d'implémentations concrètes ou de mocks lors des tests. Par exemple, dans Program.cs, la chaîne d'injection complète s'établit clairement : création

du JsonLogger, injection dans FileTransferService, injection dans BackupManager, injection finale dans ConsoleUI, permettant de remplacer n'importe quel maillon (logger JSON par logger XML) sans recompilation du code métier.

3.2.4 Repository Pattern (BackupManager)

Le BackupManager implémente le pattern Repository en agissant comme une couche d'abstraction entre la logique métier et la persistance des jobs dans le fichier jobs.json. Cette classe centralise toutes les opérations CRUD via des méthodes explicites (CreateJob(), GetJob(), GetJobByName(), GetAllJobs(), DeleteJob(), ModifyJob(), SaveJob()), masquant complètement les détails techniques de sérialisation JSON et de gestion du système de fichiers. Les méthodes privées LoadJobsFromFile() et SaveJobsToFile() encapsulent la logique de persistance, permettant de remplacer ultérieurement le stockage JSON par une base de données SQL sans impacter les 1790 lignes de code de l'interface utilisateur. Le repository gère également la logique métier (validation de la limite de 5 jobs, unicité des noms, synchronisation via LoadJobs() après chaque SaveJob()) et garantit l'intégrité des données à travers l'utilisation de fichiers temporaires lors des écritures.

3.2.5 Observer Pattern (StateWriter)

Le StateWriter implémente implicitement le pattern Observer en surveillant et enregistrant en temps réel l'état des jobs de sauvegarde dans le fichier state.json via la méthode UpdateJobState(BackupJob job). À chaque étape significative d'un backup (changement de progression, modification du fichier en cours via job.SetCurrentFile(), achèvement via job.MarkAsCompleted(), erreur via job.MarkAsError()), le FileTransferService et le BackupManager notifient le StateWriter qui sérialise immédiatement l'état complet du job. Cette architecture permet à de futurs observateurs (interface graphique .NET MAUI pour monitoring temps réel, API REST pour supervision distante prévue en version 2.0) de s'abonner aux changements d'état en lisant simplement state.json à intervalles réguliers, sans nécessiter de modification du code de sauvegarde existant ni de mécanisme complexe de publish-subscribe.

3.2.6 Factory Method Pattern (GetFormatter dans LoggerBase)

La méthode GetFormatter<T>() de LoggerBase implémente le pattern Factory Method en déléguant la création d'instances de formatters selon l'énumération LogFormat (_format)

configurée, retournant soit un `JsonFormatter<T>` (avec options `prettyPrint: true`, `paginate: true`), soit un `XmlFormatter<T>` (avec option `indent: true`). Cette fabrique consulte d'abord le dictionnaire `_formatters` pour réutiliser les instances déjà créées (optimisation performance), puis instancie et enregistre le formatter approprié si nécessaire, centralisant ainsi toute la logique de décision. L'ajout d'un nouveau format (CSV, binaire) nécessiterait uniquement l'extension de l'énumération `LogFormat`, la création de la classe `CsvFormatter<T>` implémentant `ILogFormatter<T>`, et l'ajout d'un case dans le switch de `GetFormatter<T>()`, sans modification des lignes de logique métier dans `LoggerBase` ni des classes clientes (`BackupManager`, `FileTransferService`).

3.3 Bibliothèque EasyLog.dll (documentation et rétrocompatibilité)

Conformément à la stratégie de ProSoft visant à mutualiser les développements, la bibliothèque `EasyLog.dll` a été conçue comme un composant autonome destiné à être intégré dans de multiples projets futurs, au-delà du seul périmètre d'EasySave. En conséquence, elle est livrée avec sa propre documentation technique détaillée, distincte de celle du projet principal. Cette documentation exhaustive décrit les signatures des méthodes publiques, les types de données attendus et les codes d'erreur retournés, permettant à toute autre équipe de développement de l'entreprise d'implémenter cette solution de journalisation sans avoir besoin d'analyser son code source interne.

La stabilité de l'interface de programmation (API) exposée par la DLL. Nous nous sommes engagés à ce que la signature des méthodes ne change pas lors des futures mises à jour, garantissant ainsi une rétrocompatibilité totale avec la version 1.0 du logiciel, comme exigé par le cahier des charges. Cette contrainte d'architecture assure que toute optimisation interne ou ajout de fonctionnalité au sein d'EasyLog n'entraînera aucune régression ni nécessité de refactoring pour les applications clientes existantes, sécurisant ainsi la maintenance à long terme du parc applicatif ProSoft.

3.4 Gestion des données (System.Text.Json et structuration des fichiers dans %AppData%)

La manipulation des données structurées repose intégralement sur la bibliothèque native System.Text.Json intégrée au framework .NET 8. Nous avons privilégié cette solution technologique au détriment de bibliothèques tierces historiques pour sa légèreté et ses performances supérieures en termes d'allocation mémoire. Ce choix est déterminant pour garantir la fluidité de l'application, notamment lors de l'écriture à haute fréquence des journaux d'événements exigée par le cahier des charges, tout en assurant une conformité stricte aux standards JSON pour l'interopérabilité avec les futurs outils de la suite ProSoft.

En ce qui concerne le stockage physique des fichiers, nous avons interdit toute écriture à la racine du disque système ou dans des répertoires temporaires génériques, conformément à l'interdiction formelle des emplacements de type "C:\temp\" spécifiée dans les contraintes du projet. L'ensemble des fichiers de configuration, les logs journaliers et le fichier d'état en temps réel sont systématiquement stockés dans le dossier AppData\Roaming\EasySave du profil utilisateur. Ce choix stratégique permet de garantir que l'application puisse écrire ses données sur les serveurs des clients sans nécessiter de droits d'administrateur, respectant ainsi les politiques de sécurité standard des environnements Windows et assurant la persistance des données utilisateur indépendamment de la machine utilisée.

3.5 Internationalisation (gestion du bilinguisme FR/EN dans la console)

Pour répondre à l'exigence de bilinguisme (Français/Anglais) imposée dès la version 1.0, tout en respectant la contrainte de rédaction du code source exclusivement en anglais, nous avons rejeté l'usage de chaînes de caractères codées en dur (*hard-coded strings*) au profit d'un mécanisme de ressources basé sur des fichiers JSON. Cette approche repose sur l'utilisation d'un fichier de configuration externe qui agit comme un dictionnaire de traduction, séparant strictement la logique métier du contenu textuel destiné à l'utilisateur.

Concrètement, l'application ne manipule que des clés d'identification abstraites dans le code C#. Au démarrage, un module de gestion de la langue interroge la configuration utilisateur ou la langue du système d'exploitation afin de charger dynamiquement les valeurs correspondantes depuis le fichier JSON approprié (français ou anglais). Cette architecture découplée offre une grande flexibilité : elle permet de modifier ou d'affiner les traductions sans recompilation de l'exécutable, et rend l'application immédiatement extensible à de nouvelles langues en ajoutant simplement de nouvelles sections ou fichiers JSON respectant la structure standardisée.

4. Spécifications fonctionnelles (V1.0)

4.1 Gestion des travaux de sauvegarde (Limitation à 5, paramètres Source/Cible/Type)

La version 1.0 d'EasySave permet à l'utilisateur de définir et de gérer des travaux de sauvegarde, avec une limitation volontaire à cinq tâches simultanément configurables. Cette contrainte, imposée par le cahier des charges, vise à encadrer l'usage du logiciel tout en garantissant une gestion simple et maîtrisée des ressources lors de l'exécution des sauvegardes.

Chaque travail de sauvegarde est caractérisé par un nom explicite, un ou plusieurs répertoires source (5 maximums), un répertoire cible et un type de sauvegarde. Le répertoire source correspond à l'emplacement des données à sauvegarder, tandis que le répertoire cible définit l'emplacement de destination des fichiers copiés. Ces répertoires peuvent être situés sur des disques locaux, des supports externes ou des lecteurs réseaux, ce qui permet une grande flexibilité d'utilisation dans des contextes variés, aussi bien en environnement personnel que professionnel.

Deux types de sauvegarde sont proposés. La sauvegarde complète consiste à copier l'intégralité des fichiers et sous-répertoires présents dans le répertoire source vers le répertoire cible. La sauvegarde différentielle, quant à elle, ne transfère que les fichiers ayant été modifiés depuis la dernière sauvegarde (complète / différentielle), permettant ainsi d'optimiser le temps d'exécution et l'utilisation de l'espace disque. Le choix du type de sauvegarde est effectué lors de la création ou de la modification du travail et conditionne le comportement du processus de copie lors de l'exécution.

L'utilisateur dispose de fonctionnalités lui permettant de créer, consulter, modifier ou supprimer des travaux de sauvegarde à tout moment. Les paramètres de chaque tâche sont persistés afin de garantir leur disponibilité entre deux exécutions de l'application. Ce mode de gestion assure une expérience cohérente et reproductible, tout en facilitant la maintenance et l'évolution future du logiciel.

4.2 Exécution et lignes de commande (arguments 1-3, 1;3)

EasySave version 1.0 peut être exécuté de manière interactive via un menu console, mais également de manière automatisée grâce à l'utilisation d'arguments passés en ligne de commande. Cette fonctionnalité répond à un besoin d'intégration du logiciel dans des scripts, des tâches planifiées ou des environnements serveur, où une interaction utilisateur directe n'est pas souhaitée.

Deux syntaxes d'exécution sont prises en charge. La première permet de spécifier une plage de travaux à exécuter, à l'aide d'un intervalle numérique, par exemple " 1-3 ". Dans ce cas, le logiciel exécute séquentiellement l'ensemble des travaux compris entre les indices indiqués. La seconde syntaxe permet de sélectionner explicitement des travaux non consécutifs, à l'aide d'un séparateur, par exemple " 1;3 ". Cette approche offre une flexibilité dans le choix des sauvegardes à lancer, sans nécessiter de modification préalable de la configuration.

Lors du lancement via la ligne de commande, le logiciel analyse et valide les arguments fournis afin de s'assurer de leur cohérence avec les travaux effectivement configurés. Les indices inexistantes ou incorrects sont ignorés ou signalés, garantissant un comportement robuste et prévisible. Une fois les arguments validés, les travaux sélectionnés sont exécutés de manière séquentielle, dans le respect de l'ordre défini par la syntaxe utilisée.

Quel que soit le mode d'exécution, chaque sauvegarde déclenchée génère en temps réel des informations de suivi et de journalisation, assurant une traçabilité complète des opérations réalisées. Cette cohérence entre exécution interactive et exécution automatisée garantit une utilisation homogène du logiciel et prépare son intégration dans des environnements professionnels plus complexes.

4.3 Journalisation et état en temps réel (Format des fichiers JSON, lisibilité Notepad)

La version 1.0 d'EasySave intègre un mécanisme complet de journalisation et de suivi en temps réel des sauvegardes, conformément aux exigences du cahier des charges. Deux types de fichiers distincts sont produits : un fichier de log journalier retraçant l'ensemble des actions réalisées, et un fichier d'état unique décrivant en continu l'avancement des travaux de sauvegarde.

Le fichier de log journalier enregistre, en temps réel, chaque action effectuée lors de l'exécution des sauvegardes, notamment la création de répertoires et le transfert des fichiers. Chaque entrée de log contient a minima un horodatage précis, le nom du travail de sauvegarde concerné, les chemins complets des fichiers source et destination au format UNC, la taille du fichier traité ainsi que le temps de transfert exprimé en millisecondes. En cas d'erreur, ce temps est enregistré avec une valeur négative, permettant d'identifier rapidement les incidents.

En parallèle, un fichier d'état unique est maintenu et mis à jour en continu tout au long de l'exécution des sauvegardes. Ce fichier reflète à tout instant l'état courant de chaque travail, qu'il soit actif ou inactif. Lorsqu'un travail est en cours, il fournit des informations détaillées telles que le nombre total de fichiers à traiter, la taille totale des données à transférer, le pourcentage d'avancement, le nombre et la taille des fichiers restants, ainsi que les chemins complets du fichier source en cours de traitement et de sa destination.

Les fichiers de log et d'état sont tous deux générés au format JSON. Ce choix garantit une structure de données standardisée, facilement exploitable par d'autres outils ou applications, tout en restant lisible par un utilisateur ou un technicien de support. Afin de permettre une consultation rapide via un éditeur de texte simple tel que Notepad, les fichiers sont volontairement formatés avec des retours à la ligne et une indentation claire, assurant une lecture fluide sans outil spécifique. Cette organisation peut être assimilée à une pagination logique, où chaque entrée ou état est clairement identifiable, ce qui facilite le diagnostic et l'analyse des sauvegardes.

Enfin, les emplacements de stockage de ces fichiers ont été définis de manière à être compatibles avec une utilisation sur des postes clients et des serveurs, en évitant les

répertoires temporaires non persistants. Ce choix garantit la pérennité des informations de suivi et de journalisation, essentielles tant pour l'utilisateur final que pour le support technique.

5. Conception (Diagrammes UML)

La conception UML de la version 1.0 d'EasySave a pour objectif de formaliser une architecture claire, modulaire et évolutive, tout en répondant strictement aux exigences du cahier des charges. Les différents diagrammes produits constituent outil de conception permettant de justifier les choix techniques réalisés, notamment la séparation des responsabilités, la maintenabilité du code et la préparation des évolutions futures. Cette section présente et commente les principaux diagrammes UML réalisés pour cette première version.

5.1 Diagramme de cas d'utilisation (Vue macro des fonctionnalités)

Le diagramme de cas d'utilisation fournit une vision globale et fonctionnelle du système EasySave du point de vue de l'utilisateur. Il met en évidence les interactions possibles entre l'utilisateur et l'application, sans entrer dans les détails techniques d'implémentation.

Dans la version 1.0, l'utilisateur est l'unique acteur du système. Il peut créer, visualiser, modifier ou supprimer des tâches de sauvegarde, ce qui correspond aux opérations de gestion essentielles attendues pour un logiciel de sauvegarde. L'exécution d'une tâche s'accompagne automatiquement de l'ajout d'informations dans le fichier de log journalier. Cette relation est représentée par une dépendance de type "include" traduisant le fait que toute exécution de sauvegarde entraîne obligatoirement une écriture dans le système de journalisation.

Le diagramme prend également en compte l'exécution du logiciel via le terminal. Les deux modes d'exécution sont intégrés dès la conception : l'exécution par plage de tâches (par exemple "1-3") ainsi que l'exécution par sélection explicite de tâches (par exemple "1;3").

Le diagramme intègre également le choix de la langue de l'application, répondant à la contrainte de support du français et de l'anglais dès la version initiale. Certains cas d'utilisation apparaissent volontairement comme étant hors périmètre de la version 1.0, mais sont néanmoins représentés afin d'anticiper les évolutions futures. Le code couleur utilisé permet d'identifier clairement les fonctionnalités introduites dans les versions ultérieures

(1.1, 2.0 et 3.0), ce qui inscrit la conception dans une logique de cycle de vie logiciel et facilite la compréhension de la trajectoire d'évolution du produit.

5.2 Diagramme de classes (Structure de l'application et de la DLL)

Le diagramme de classes décrit la structure statique de l'application EasySave et met en évidence la séparation nette entre la logique métier de sauvegarde et la logique de journalisation.

La logique de sauvegarde repose principalement sur les classes représentant les travaux de sauvegarde. Chaque tâche de sauvegarde est modélisée par une classe dédiée contenant les informations nécessaires à son identification, à son exécution et à son suivi. La gestion globale des tâches est assurée par un composant central, le BackupManager, chargé de créer, stocker, exécuter et limiter le nombre de travaux conformément aux contraintes de la version 1.0. Ce composant implémente un pattern Repository, encapsulant complètement la sérialisation et la désérialisation JSON des jobs dans le fichier *jobs.json*, garantissant ainsi une isolation stricte entre la logique métier et la couche de persistance. Cette abstraction permet d'envisager un changement futur du mécanisme de stockage (base de données SQL, stockage cloud) sans impacter l'interface utilisateur existante.

La responsabilité du transfert effectif des fichiers est isolée dans des classes spécialisées, ce qui permet d'éviter toute dépendance directe entre l'interface utilisateur et le système de fichiers. Les chemins manipulés sont normalisés au format UNC, garantissant la compatibilité avec les disques locaux, les supports externes et les lecteurs réseaux, conformément au cahier des charges. La différenciation entre sauvegarde complète et sauvegarde différentielle repose sur une stratégie clairement identifiée, correspondant à une implémentation du Strategy Pattern. Ce choix permet de sélectionner dynamiquement le comportement de sauvegarde sans modifier la structure globale du code, limitant la duplication de logique et facilitant l'ajout de nouveaux types de sauvegarde dans les versions futures.

La journalisation est entièrement déportée dans la DLL EasyLog, qui repose sur des interfaces génériques définissant un contrat clair pour l'écriture des logs, indépendamment du format ou du contexte d'utilisation. Les composants de journalisation et d'écriture d'état

(StateWriter) sont conçus de manière thread-safe grâce à l'utilisation de mécanismes de verrouillage (*lock*), garantissant la cohérence des écritures concurrentes dans *state.json* et les fichiers de logs, ce qui constitue un prérequis critique pour l'implémentation des sauvegardes parallèles prévues en version 3.0.

L'interface utilisateur ConsoleUI est décomposée en dix classes spécialisées (écrans, assistants, helpers et coordinateur), conformément au principe de responsabilité unique (SRP). Cette modularisation remplace un monolithe initial et améliore significativement la maintenabilité et la testabilité unitaire de chaque composant fonctionnel (création, modification, suppression, exécution et affichage des travaux).

Toutes les classes principales reçoivent leurs dépendances par constructeur, appliquant le principe d'inversion de dépendances (SOLID – D) et permettant l'injection de mocks pour les tests. Cette architecture favorise également le remplacement d'implémentations sans recompilation, par exemple pour substituer un logger JSON par un logger XML.

Les règles métier sont centralisées dans le BackupManager, qui assure la validation de l'intégrité des données (limitation à cinq jobs, unicité des noms, validation des chemins). Cette centralisation garantit que les contraintes métier sont respectées indépendamment du point d'entrée (interface console, arguments en ligne de commande, ou future API REST). La gestion des erreurs est hiérarchisée : les erreurs I/O sont capturées au niveau du FileTransferService, enrichies et propagées par le BackupManager, puis traduites et affichées par l'interface utilisateur via le service de localisation. Cette séparation évite tout couplage entre les exceptions techniques et la présentation utilisateur.

Le support de l'internationalisation est assuré par un LocalizationService centralisant toutes les traductions dans un fichier JSON unique. L'ensemble des classes utilise une méthode d'accès unifiée aux ressources linguistiques, garantissant la cohérence des messages et permettant l'ajout d'une nouvelle langue sans modification du code source.

Enfin, l'architecture globale respecte le modèle MVVM (Model–View–ViewModel), garantissant une séparation claire des responsabilités entre la présentation, la logique applicative et les entités métier.

La View correspond à la couche de présentation, actuellement basée sur *Terminal.Gui*. Elle se limite à l'affichage et à la gestion des interactions utilisateur, sans contenir de logique métier.

La ViewModel regroupe la logique applicative et les services, incluant notamment les composants de gestion des sauvegardes ainsi que la DLL EasyLog. Cette couche assure le rôle d'intermédiaire entre la vue et les modèles, expose les commandes et propriétés nécessaires à l'interface, et encapsule l'ensemble des règles de gestion, garantissant ainsi l'absence de dépendance directe entre la View et la logique métier.

Les Models représentent les entités métier (jobs de sauvegarde, états, configurations) et sont totalement indépendants de toute interface utilisateur. Ils constituent le cœur du domaine applicatif et peuvent être réutilisés dans d'autres contextes, comme une future API REST ou une interface graphique .NET MAUI.

Cette conformité au modèle MVVM facilite l'évolutivité de l'application, améliore la testabilité (en permettant le test isolé des ViewModels et des Models) et prépare une transition transparente vers une interface graphique dans les versions ultérieures, sans modification de la logique métier.

Cette séparation garantit qu'une migration vers une interface graphique .NET MAUI en version 2.0 nécessitera uniquement le remplacement de la couche View. L'architecture modulaire favorise également une testabilité par conception, permettant des tests unitaires, d'intégration et de bout en bout avec une granularité fine, chaque classe étant limitée à une taille raisonnable facilitant la maintenance et l'évolution du projet.

5.3 Diagramme de séquence (Focus sur le processus de sauvegarde et logs en temps réel)

Les diagrammes de séquence permettent de décrire le comportement dynamique du système lors de l'exécution d'une sauvegarde. Ils illustrent les échanges entre l'utilisateur, l'interface console, les services métiers et les composants techniques tels que le logger et le gestionnaire de l'état.

Le scénario principal débute par une action de l'utilisateur, soit via le menu interactif, soit par la ligne de commande. La demande d'exécution est transmise à la couche de gestion des

sauvegardes, qui initialise le traitement du travail sélectionné. Dès le démarrage, les informations globales nécessaires au suivi sont calculées, puis le fichier d'état est mis à jour afin de refléter le passage de la tâche à l'état actif. L'écriture de ce fichier, tout comme celle de la configuration, s'effectue dans un répertoire applicatif de type %AppData%, permettant un emplacement persistant et adapté à une utilisation en environnement client ou serveur.

Pour chaque fichier traité, la séquence met en évidence un enchaînement systématique et cohérent : sélection du fichier, copie vers le répertoire cible, mesure du temps de transfert, mise à jour de la progression, puis écriture d'une entrée dans le fichier de log journalier via la DLL EasyLog. Cette séquence se répète jusqu'à la fin du traitement, garantissant une journalisation en temps réel et une visibilité continue de l'avancement. En cas d'exception lors du transfert, celle-ci est interceptée et un temps négatif est enregistré.

En cas d'erreur, le diagramme montre que celle-ci est capturée, consignée dans le log avec un temps négatif, puis intégrée dans l'état du travail de sauvegarde. Ce mécanisme assure la traçabilité des incidents tout en permettant à l'application de terminer proprement son exécution.

5.4 Diagramme d'activité (Logique de l'exécution séquentielle)

Le diagramme d'activité décrit la logique globale de l'exécution séquentielle des sauvegardes, telle qu'imposée par la version 1.0. Il permet de visualiser le flux de contrôle depuis le lancement de l'application jusqu'à la fin complète des traitements.

Après l'initialisation et le chargement de la configuration, le système détermine si une exécution automatique est demandée via la ligne de commande ou si l'utilisateur souhaite interagir avec le menu. Dans le cas d'une exécution séquentielle de plusieurs tâches, celles-ci sont traitées l'une après l'autre, ce qui simplifie la gestion des ressources et garantit un comportement prévisible.

Pour chaque tâche, le flux inclut une étape de décision permettant de distinguer le traitement d'une sauvegarde complète d'une sauvegarde différentielle, notamment par la vérification des fichiers modifiés. Le diagramme détaille ensuite les étapes successives : analyse du répertoire source, création de l'arborescence cible, traitement fichier par fichier, mise à jour continue du fichier d'état et délégation de l'écriture des logs à la DLL EasyLog.

Ainsi, toute évolution de la DLL EasyLog peut être réalisée sans remettre en cause le fonctionnement global du logiciel. Le diagramme met enfin en évidence les points de sortie du flux, que ce soit en fin de traitement ou en cas d'erreur critique, et constitue une référence claire pour l'évolution vers des modes d'exécution plus complexes dans les versions ultérieures, notamment l'exécution parallèle prévue en version 3.0.

6. Table des figures

Figure 1 : Trello du projet, avec les cartes usuelles (To Do, In Progress, To Verify et Done) mais dans le To Do, ne figurent que les livrables de la phase 1 (V1 et V1.1), des étiquettes “APP” pour la tâche de développement et “DOC” pour la rédaction des documents (utilisateurs, commentaires) et “UML” pour les diagrammes. Les tâches de la V2 ont été ajoutées en anticipation de la suite.....	12
Figure 2 : Diagramme de Gantt du Livrable 1, en bleu l’avancement du projet conforme à nos attentes, en violet un événement imprévu nous ayant amené revoir et paralléliser une tâche.....	13
Figure 3 : Tableau de chiffrage des coûts pour l’entreprise cliente.....	14

7. Glossaire

API (Application Programming Interface) : Interface de programmation permettant à des composants logiciels distincts de communiquer entre eux. Dans le cadre d'EasySave, elle fait référence aux méthodes exposées par la bibliothèque EasyLog.dll utilisables par l'application principale.

CLI (Command Line Interface) : Interface en ligne de commande permettant à l'utilisateur d'interagir avec le logiciel via des commandes textuelles, par opposition à une interface graphique (GUI).

Design Pattern (Patron de conception) : Solution standardisée et éprouvée répondant à un problème récurrent de conception logicielle. EasySave implémente notamment les patterns Singleton, Strategy, Repository et Observer.

DLL (Dynamic Link Library) : Bibliothèque de liens dynamiques contenant du code compilé (classes, méthodes) pouvant être utilisé par plusieurs programmes. Le projet utilise EasyLog.dll pour mutualiser la logique de journalisation.

DRY (Don't Repeat Yourself) : Principe fondamental de développement logiciel visant à réduire la répétition d'informations ou de code. L'objectif est de centraliser la logique pour faciliter la maintenance et réduire les risques d'erreurs.

GitFlow : Modèle de gestion de versions pour Git, structurant le développement autour de branches spécifiques (main, develop, feature) pour organiser le travail collaboratif et les cycles de livraison.

IDE (Integrated Development Environment) : Environnement de développement intégré. Logiciel rassemblant les outils nécessaires au développement (éditeur de code, compilateur, débogueur). L'équipe utilise Visual Studio 2022.

I/O (Input/Output) : Opérations d'entrées/sorties. Désigne ici les échanges de données entre le processeur, la mémoire et les périphériques de stockage (lecture/écriture de fichiers lors des sauvegardes).

JSON (JavaScript Object Notation) : Format standard léger d'échange de données textuelles. Il est utilisé par EasySave pour structurer les fichiers de configuration, les journaux d'événements (logs) et les états en temps réel.

LTS (Long Term Support) : Version d'un logiciel ou d'un framework (comme .NET 8.0) bénéficiant d'un support technique et de mises à jour de sécurité sur une période étendue, garantissant la stabilité pour les entreprises.

MVVM (Model-View-ViewModel) : Patron d'architecture logicielle séparant l'interface utilisateur (Vue), la logique de présentation (ViewModel) et les données (Modèle). Cette structure facilite les tests et l'évolution vers des interfaces graphiques (WPF/MAUI).

NuGet : Gestionnaire de paquets officiel pour la plateforme .NET, permettant de télécharger et d'intégrer facilement des bibliothèques tierces (comme System.Text.Json) dans le projet.

Refactoring : Processus de restructuration du code source existant visant à améliorer sa structure interne, sa lisibilité ou ses performances, sans modifier son comportement fonctionnel externe.

ROI (Return On Investment) : Retour sur investissement. Indicateur financier permettant de mesurer la rentabilité du projet en comparant les coûts de développement aux revenus générés par la vente des licences et la maintenance.

Singleton : Patron de conception garantissant qu'une classe ne possède qu'une seule instance et fournissant un point d'accès global à celle-ci. Utilisé dans EasySave pour la gestion centralisée des logs et de la configuration.

UML (Unified Modeling Language) : Langage de modélisation graphique standardisé utilisé pour visualiser, spécifier, construire et documenter les éléments d'un système logiciel (diagrammes de classes, séquences, etc.).

UNC (Universal Naming Convention) : Convention de nommage standard pour identifier les partages réseaux (ex: \\Serveur\Partage\Fichier), assurant la compatibilité des sauvegardes sur le réseau.

8. Annexes

8.1 Guide d'installation et Support Technique (Détails ProSoft)

Listez l'emplacement des fichiers, la configuration minimale et le manuel d'une page.

8.2 Lien vers le dépôt GitHub

<https://github.com/MatheoCarval/Projet-EasySave>